

SALON SaaS PLATFORM

Document purpose: This document explains what you are expected to build, how you must collaborate, and what you must submit for completion of the internship final project.

1. Project Objective

You will work as part of a software development team to build one integrated Salon SaaS Platform. The project is divided across three teams so that development can happen simultaneously. Each student owns specific modules, but the final result must operate as one connected product.

- Understand and implement requirements from the existing PRD & SRS.
- Develop production-style React or Flutter functionality.
- Work with a shared REST API and database-backed application.
- Use Git and GitHub correctly in a collaborative team environment.
- Test, integrate, debug, and deploy your own work.
- Demonstrate and explain your individual contribution in the final assessment.

2. Important Rule: Work in Parallel

The three teams must develop at the same time. You must not wait for another team to finish its complete project before beginning your work.

If a dependency is not ready, continue development using the agreed API contract, mock data, fixtures, or a temporary stub. Replace the temporary implementation with the real integration when the dependency becomes available.

The objective is to simulate a real software team, where multiple developers work concurrently on different parts of the same system.

3. Teams and Responsibilities

Team	Members	Primary Area
React Group A	Abdullah Mapari; Apurva Dinesh Nikam; Chetan Hemant Wagh; Surabhi Digambar Bhadkamkar	Core web operations and business transactions
React Group B	Muhammed Husain Kagdi; Nikit Suthar; Preet Avinash Patil; Sanika Nilesh Kadam	Services, staff, inventory, reporting/engagement
Flutter Group	Ketan Narendra Rajput; Shreya Sandip Kadam	Customer-facing mobile application

4. React Group A — Individual Assignments

A1 — Abdullah Mapari

Module: Platform Foundation, Authentication, Role-Based Access and Dashboard

- Implement login/logout/session handling and protected routes.
- Implement role-aware navigation and access control required by the existing PRD.
- Implement the main web dashboard and its assigned metrics/cards.
- Provide reusable foundation components required by the assigned web modules.
- Integrate with the approved authentication API.

A2 — Apurva Dinesh Nikam

Module: Customer / CRM and Customer Membership/Loyalty View

- Customer list, search/filter, create, edit, profile and status.
- Customer history required by the PRD.
- Staff-facing membership/loyalty information.
- Customer-related API integration and validation.

A3 — Chetan Hemant Wagh

Module: Appointments, Calendar and Queue Management

- Appointment creation, editing, cancellation and status.
- Calendar views and appointment scheduling.
- Queue management.
- Customer/service/staff relationships.
- Validation of appointment states and API integration.

A4 — Surabhi Digambar Bhadkamkar

Module: Billing, POS, Invoicing and Payment Records

- Billing/POS interface.
- Invoice creation and viewing.
- Line items, taxes/discounts and totals according to the PRD.
- Payment status and billing history.
- Integration with customers, services and appointments.

5. React Group B — Individual Assignments

B1 — Muhammed Husain Kagdi

Module: Services, Categories, Packages and Pricing

- Service categories and service CRUD.
- Pricing, duration and status.
- Packages where defined by the PRD.
- Search/filter and validation.
- Provide service data for appointment and billing integrations.

B2 — Nikit Suthar

Module: Staff Management, Scheduling, Attendance and Leave

- Staff records, profiles and roles/designations.
- Working schedules and availability.
- Attendance and leave.
- Authorization for staff administration.
- Integration with appointment scheduling.

B3 — Preet Avinash Patil

Module: Inventory, Products, Stock and Inventory Transactions

- Product/item master and categories.
- Stock quantity and stock-in/stock-out.
- Inventory transaction history.
- Low-stock information.
- Supplier/vendor functionality where included in the existing PRD/SRS.
- Integration with other modules where required.

B4 — Sanika Nilesh Kadam

Module: Reports, Analytics, Marketing, Notifications and Business Insights

- Operational reports and relevant filters.
- Business/customer/appointment summaries defined by the PRD.
- Assigned marketing/campaign functionality.
- Notification-related web interfaces.
- Report/export functionality where specified.
- Consume real integrated data rather than creating duplicate reporting data.

6. Flutter Group — Customer Mobile Application

The Flutter team will build a focused customer-facing mobile application. It must be simple enough to complete successfully but must use the same backend and real application data. It is not a separate demo project and it is not a duplicate of the administration portal.

F1 — Ketan Narendra Rajput

Module: Flutter Foundation, Authentication, Customer Profile and Service Browsing

- Flutter project structure, navigation and reusable components.
- Login/logout and session handling.
- Customer profile viewing/editing where supported.
- Browse salon services, categories and service details.
- Shared API client and loading/error/empty states.
- Integration with the common authentication and service APIs.

F2 — Shreya Sandip Kadam

Module: Appointment Booking, My Appointments and Customer Engagement

- Service selection and appointment booking.

- Date/time selection using available API data.
- Booking confirmation.
- Upcoming and previous appointments.
- Appointment details and supported cancellation/rescheduling.
- Basic membership/loyalty visibility.
- Customer notification UI where supported.
- Integration testing with the shared backend.

7. Flutter Scope — What You Do Not Need to Build

- Super Admin.
- Staff administration.
- Payroll.
- Complex inventory administration.
- POS administration.
- Advanced reporting and business analytics.
- Complex salon configuration.

The Flutter application should concentrate on a useful customer journey: authenticate → browse services → book an appointment → view appointments → view relevant customer information.

8. Shared Product Integration

All groups are building one system. The same backend and database must be used for the integrated product.

Flow	Expected Integration
Customer	React customer module provides customer data used by appointments and billing; Flutter uses customer-facing APIs.
Services	React service module provides service data consumed by appointments, billing and Flutter browsing/booking.
Appointments	React appointment module manages salon-side appointments; Flutter creates/views customer appointments through the API.
Staff	Staff availability can be used by appointment scheduling.
Billing	Billing consumes valid customer, service and appointment information.
Reports	Reports consume integrated business data from the actual modules.

9. GitHub Collaboration Rules

1. Work only in the shared project repository provided for the assessment.
2. Do not push directly to main.
3. Do not push directly to develop unless the project maintainer explicitly permits it.
4. Create a feature branch for your work using feature/<group>/<module> naming.
5. Keep commits focused and meaningful.
6. Open a Pull Request when a logical unit is ready for review.
7. Another team member must review your Pull Request.
8. Resolve review comments before merge.
9. Do not commit passwords, API keys, database credentials, tokens or secret files.
10. Do not delete or rewrite another student's module simply to make your own implementation easier.

10. API and Dependency Rules

When your module depends on another module, agree on the API contract instead of waiting for implementation.

- Define endpoint and HTTP method.
- Define request and response structures.
- Define validation and error behavior.
- Use mock/fixture data when the dependency is not yet available.
- Do not create duplicate APIs for an operation that already has an approved contract.
- Keep the existing project architecture and approved technology stack.

11. Testing Requirements

A screen that renders is not considered complete. Your module must work correctly with its data and expected business behavior.

- Test the normal/happy path.
- Test validation and invalid input.
- Test expected API errors and empty states.
- Test authorization/permissions where applicable.
- Test responsive behavior for React.
- Test on an Android emulator or physical device for Flutter.
- Test your module after integration with dependent modules.

12. Deployment

The assessment environment will initially use simple cloud deployment so that students can focus on development and integration.

- React web application: Vercel assessment deployment.
- Backend/API: Render assessment deployment.
- Database: dedicated assessment database.
- Flutter: working APK/release build for final assessment.

Use environment variables for URLs and configuration. Never place credentials directly in source code. Production/company infrastructure is not part of the student environment unless explicitly authorized.

13. Definition of Done

Your assigned module is complete only when all applicable conditions below are satisfied:

- Required functionality from the PRD/SRS is implemented.
- UI is complete and usable.
- API integration is complete.
- Validation is implemented.
- Loading, empty and error states are handled.
- Authorization is enforced where required.
- Data is correctly persisted and retrieved.
- Module-level testing is completed.
- Code is pushed through the required Git workflow.
- Pull Request has been reviewed and review comments resolved.
- Module works with the integrated application.

14. Final Submission

Each student must be able to demonstrate their own contribution. The final team submission is one integrated Salon SaaS Platform.

- Complete source code in the shared repository.
- Meaningful Git history showing your contribution.
- Completed assigned module(s).
- Testing evidence.
- Required documentation/setup notes.
- Working deployed web/API environment where applicable.
- Working Flutter APK for the mobile group.
- Final demonstration and technical explanation.

15. Individual Responsibility

This is a team project, but your contribution is individually assessable. You are expected to understand the code you submit and be able to explain your architecture, implementation decisions, API usage, validation, testing, and known limitations.

If another team member's module is incomplete, do not simply wait. Follow the documented integration process, use the agreed contract, communicate the dependency, and continue all work that can be completed independently.

16. What Is Expected From You

- Read the relevant sections of the existing PRD & SRS before coding.
- Understand your assigned module completely.
- Plan your work before implementation.
- Communicate API dependencies early.
- Keep your branch clean and focused.
- Review other students' Pull Requests seriously.
- Test your work before requesting integration.
- Solve your own implementation and integration problems.
- Ask for clarification when a requirement is genuinely ambiguous—not for solutions to implementation problems.
- Deliver a working, integrated feature rather than only a collection of screens.

17. Final Assessment

This project will serve as the final assessment of your internship.

Your assessment will be based on your ability to independently complete your assigned module, work effectively with your team, integrate your work with the overall Salon SaaS Platform, and demonstrate that your implementation is functional and production-ready.

You will be expected to demonstrate and explain your own work during the final assessment.

Completing only the UI or individual screens is not sufficient. Your assigned functionality must be properly implemented, connected to the required APIs and data, tested, and integrated into the final project.

The final assessment will consider both your individual contribution and the quality of the integrated project.